PROJECT SHIPWRIGHT

The Creator Studio Authoring Experience System

Governing Architecture and Product Requirements Document

Governing Principle
Creator Studio must make sophisticated Chronicle authoring feel direct, visual, reversible, discoverable, and safe. Shipwright owns how Creators manipulate canonical authoring contracts. It must never invent a second authoring truth, a second validation authority, or a second progression engine.

Version 1.0 | August 8, 2026

Repository baseline reviewed: Kgray44/treasurehuntSoT @ f1c2f22dd935322c1a71eb80c51592f243dc196d

Document Control

Field	Value
Program	Project Shipwright
Subsystem	The Creator Studio Authoring Experience System
Document type	Governing architecture and product requirements
Version	1.0
Date	August 8, 2026
Status	Governing baseline
Repository baseline reviewed	Kgray44/treasurehuntSoT at f1c2f22dd935322c1a71eb80c51592f243dc196d
Primary product surfaces	Creator Studio, Chronicle graph editor, block inspectors, libraries, preview, versioning, publishing
Core implementation rule	Shipwright owns authoring interaction and presentation; Drydock owns authoring-contract and validation truth; One Voyage owns runtime and publication truth.

This document incorporates the permanent Continuous Development and Mainline Integration doctrine established for Voyagewright: completed phases converge promptly into main, each phase is independently mainline-safe, and same-project phases do not intentionally stack on unmerged predecessors.

Contents

1. Executive Summary

Project Shipwright rebuilds Creator Studio from a powerful but increasingly dense editor into a deliberate authoring environment that ordinary Creators can learn, advanced Creators can move quickly through, and the platform can safely extend for years. The project is not a rewrite of Chronicle semantics. It is the product and interaction system that sits over the canonical Chronicle authoring model.

The current Studio is already substantial. At the verified baseline, the central TaleEditor is a very large client component that already coordinates dnd-kit pointer and keyboard sensors, undo/redo history, autosave with conflict preservation, block and chapter libraries, asset filtering, preview viewport and reduced-motion controls, version comparison, publication state, and Lanternwake publishing presentation. Shipwright preserves those successful capabilities while separating them into clearer responsibilities and improving the Creator experience.

The project therefore has two simultaneous goals: make authoring dramatically more pleasant and make the Studio architecture easier to extend without turning every future provider or block type into another conditional inside one enormous component.

Core Product Outcome

A Creator should be able to open Studio, understand the Chronicle at a glance, build or rearrange its structure visually, edit each block through purpose-built tools, see validation issues in context, preview the experience from relevant roles and devices, recover safely from mistakes, and publish with confidence. Power remains available without forcing every Creator to stare at every control all the time.

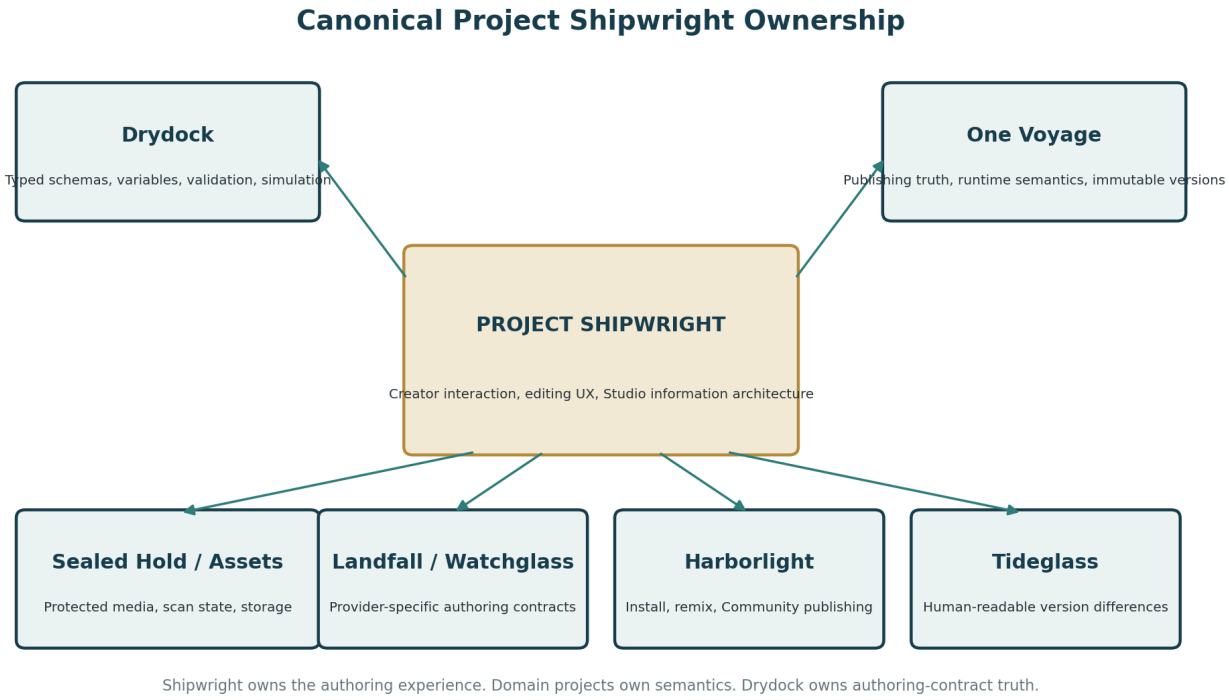

Figure 1. Shipwright owns the Creator authoring experience while consuming canonical semantics from Drydock and domain-owning projects.

2. Project Identity and Product Vision

2.1 Program Name

The subsystem is named Project Shipwright. A shipwright does not merely inspect a vessel; a shipwright shapes it, fits its parts, aligns its structure, and makes complex construction understandable through tools and craft. The name fits Creator Studio because this project governs how a Chronicle is assembled rather than how it is played.

2.2 Formal Subtitle

The Creator Studio Authoring Experience System.

2.3 Product Vision

Creator Studio should feel closer to a modern visual creative tool than a database form. Complexity should appear when it is useful, not because every capability deserves a permanently visible button. The Studio should reward exploration, preserve reversibility, explain consequences, and keep the Creator oriented inside both the narrative structure and the technical state of the Chronicle.

2.4 Primary Creator Profiles

- First-time Creator building a simple linear Chronicle without learning the platform's internal data model.
- Experienced Creator building branching, variable-driven, provider-rich Chronicles.
- Creator adapting Community content, templates, artifacts, maps, or provider assets.
- Creator maintaining a published Chronicle across multiple immutable editions.
- Creator diagnosing validation or simulation failures without leaving Studio.
- Creator using keyboard, touch, reduced motion, screen magnification, or assistive technology.

3. Current Repository Context

This governing baseline was prepared against current remote main at f1c2f22dd935322c1a71eb80c51592f243dc196d. The SHA documents what was reviewed; implementation must always fetch and revalidate current origin/main before work begins.

At this baseline, Creator Studio already contains a broad Chronicle editor. The main TaleEditor module exceeds one hundred kilobytes and owns many concerns at once: draft state, autosave, conflict state, undo/redo, drag-and-drop, chapter and block insertion, selection/focus management, asset search, locations and artifacts, preview controls, version comparison, validation state, publication state, and motion. That concentration is functional, but it raises extension cost and makes the interface feel denser as new features accumulate.

The current data shape also reflects the pre-Drydock authoring model: blocks expose configuration, presentation, completion, creator notes, enabled state, and schema version, while registry entries expose display metadata, default configuration, inspector fields, and schema version. Shipwright must consume the stricter typed contracts introduced by Drydock rather than creating independent block schemas.

Current strength	Shipwright extension
dnd-kit pointer and keyboard movement	Richer canvas, multi-select, alignment, group movement, accessible graph manipulation
Undo/redo history and autosave	Durable command model, safer recovery, clearer save/conflict state
Block/chapter library and search	Family-based library, templates, fragments, context-aware insertion
Asset/location/artifact libraries	Unified provider library architecture with usage context and richer pickers
Preview viewport and reduced-motion toggles	Role/device/mode preview workspace with Drydock scenario handoff
Version comparison data	Creator-readable history and Tideglass semantic-diff integration
Publishing presentation	Clear staged publishing workflow tied to authoritative Drydock/One Voyage results

4. Non-Negotiable Design Principles

Principle	Requirement
One authoring truth	Shipwright renders and manipulates the canonical Drydock/Chronicle contracts. It does not define competing block semantics.
Progressive disclosure	Simple tasks remain simple. Advanced controls appear when the Creator asks for them or the selected block requires them.
Reversible by default	Every ordinary edit has undo, clear consequence feedback, or an explicit confirmation where reversal is not immediate.
Graph and outline are views of one structure	No second chapter/block ordering model may emerge inside a visual canvas.
Direct manipulation plus keyboard parity	Drag, resize, connect, group, or reorder operations must have non-drag alternatives.
Validation in context	Issues appear next to the block, field, edge, asset, or setting they concern, with Drydock as authority.
No button cockpit	Permanent chrome is reserved for frequent, high-value actions. Secondary actions use contextual menus, command palette, or progressive panels.
Creator orientation	The Creator can always tell where they are, what is selected, what is saved, what is invalid, and what will happen next.
Domain-owner integration	Location, artifacts, private content, vision, Community, and other domains register authoring adapters rather than being copied into Studio.
Mainline-safe phases	Every Shipwright phase can live on main even if later phases never happen.

5. Scope and Non-Goals

5.1 Project Scope

- Creator Studio workspace information architecture and visual hierarchy.
- Chronicle graph/canvas interaction, selection, movement, grouping, organization, and navigation.
- Contextual commands, toolbars, menus, command palette, and keyboard shortcuts.
- Block-specific inspector and editor experiences built over canonical typed contracts.
- Visual authoring for choices, conditions, variables, targets, recipients, providers, and completion behavior.
- Story Block discovery, categories, templates, reusable fragments, and composition workflows.
- Integrated libraries for assets, locations, artifacts, maps, provider references, and future typed content.
- Preview, role/device/motion views, and Drydock validation/simulation presentation.
- Creator-facing version history, compare, publication preparation, and release workflow.
- Studio accessibility, responsive behavior, performance, recovery, and product polish.

5.2 Explicit Non-Goals

- Do not create a second Story Block schema registry.
- Do not create a second variable, expression, graph-validation, or simulation engine.
- Do not change One Voyage progression semantics merely to simplify Studio UI.
- Do not own Landfall location truth, Watchglass recognition truth, artifact rendering/ownership, Sealed Hold storage/cryptography, Harborlight social data, or Tideglass semantic diff truth.
- Do not implement arbitrary JavaScript or user-authored executable code.
- Do not turn Studio into a professional IDE that requires programming knowledge for ordinary Chronicles.
- Do not require drag-and-drop as the only way to perform structural edits.
- Do not silently rewrite existing published editions.
- Do not auto-apply destructive repairs without explicit Creator confirmation and undo/restore behavior.

6. Canonical Architecture and Ownership

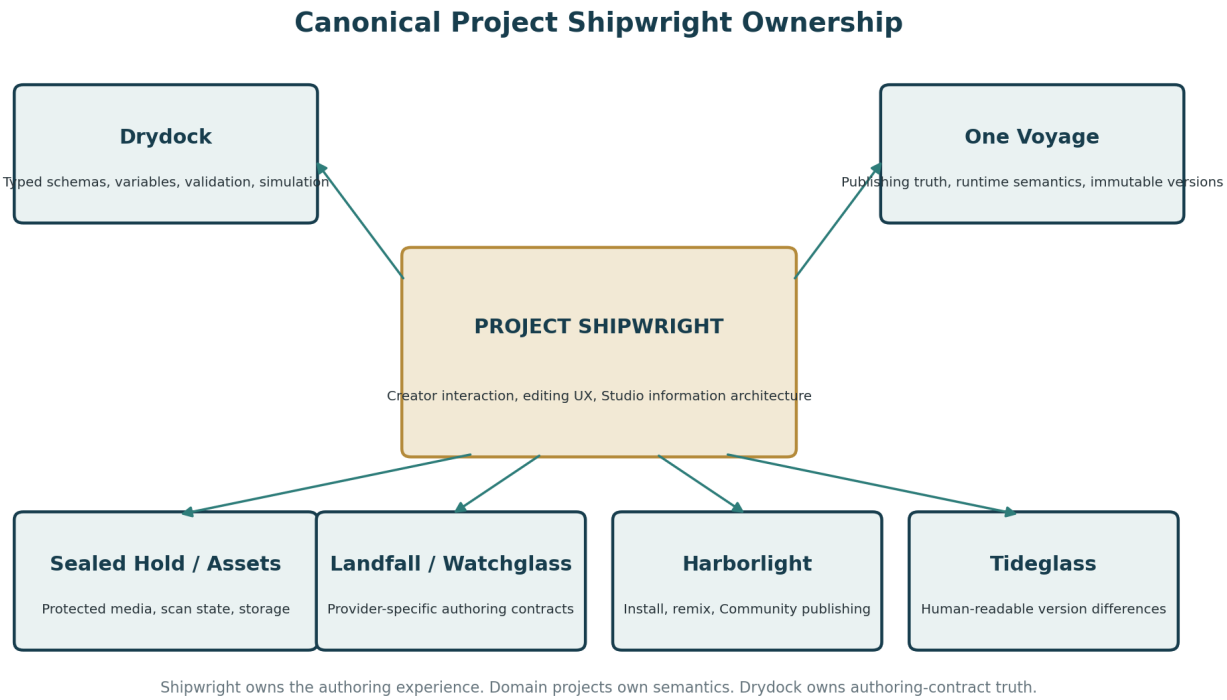

Figure 2. Canonical project boundary: Shipwright owns the human authoring layer, not the underlying semantic authorities.

System	Shipwright consumes	Shipwright must not own
Drydock	Typed block schemas, variables, expression AST, validation issues, simulation/scenario contracts, publishing evidence	Independent validation semantics or duplicate schemas
One Voyage	Draft/published/runtime contracts, immutable edition identity, authoritative publish result	Live progression, event truth, idempotency, runtime writes
Sealed Hold	Protected asset metadata, scan/quarantine state, safe private preview handles	Encryption, storage keys, scanner implementation
Harborlight	Installed content, attribution, lineage, Community publication workflow	Community listings, moderation, social graph
Landfall	Waypoint, route, region, map and provider authoring contracts	Geolocation engine or authoritative arrival
Watchglass	Vision Waypoint library/version contracts, validation/certification state	Recognition engine or evidence scoring
Artifact systems	Artifact definition/editor adapters, representation metadata	Personal ownership, grant truth, 3D runtime renderer
Tideglass	Semantic edition difference results	Independent semantic diff engine
Lanternwake	Scene contracts, motion mode, substantial presentation runtime	Animation truth or acknowledgment authority

7. Creator Studio Information Architecture

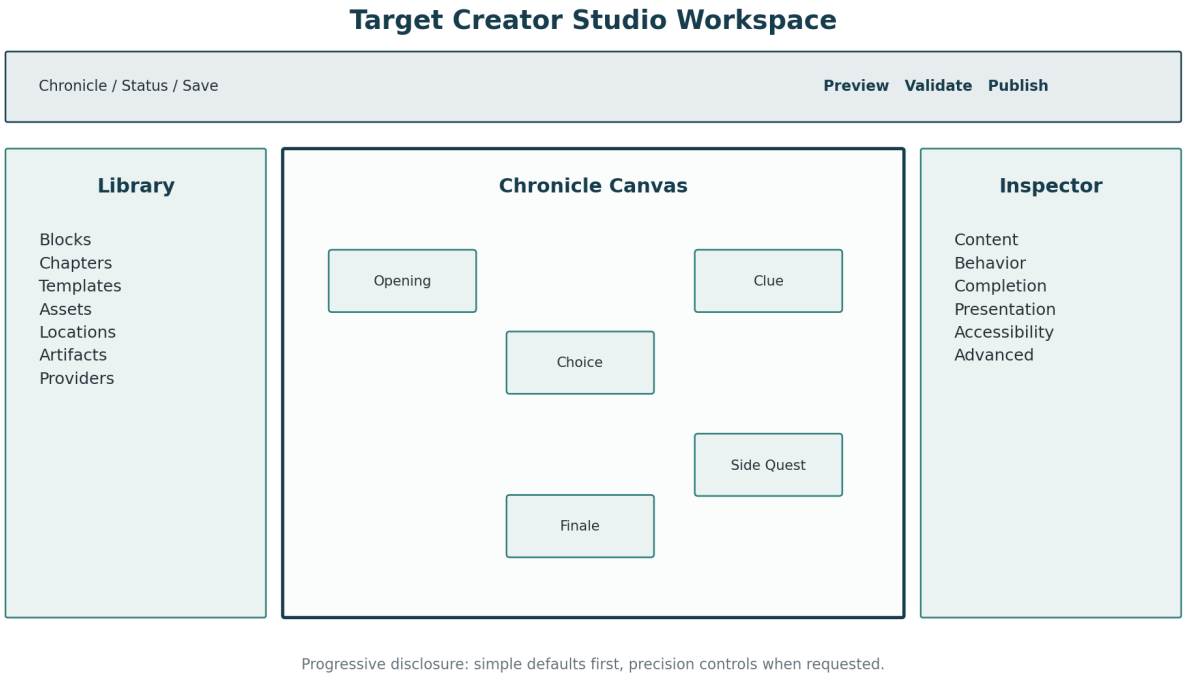

Figure 3. Target Studio workspace separates discovery, construction, and inspection without hiding important state.

The default desktop workspace should use a stable three-region authoring model: a left Library for things the Creator can add or navigate, a central Chronicle Canvas for structure, and a right Inspector for the selected object. A compact global header owns Chronicle identity, save state, validation readiness, preview, and publishing actions.

- Library: Blocks, chapters, templates/fragments, assets, locations, artifacts, maps, providers, installed Community content.
- Canvas: graph or structured outline view, zoom/pan, selection, connections, branch labels, chapter boundaries, validation badges.
- Inspector: selected block/chapter/edge properties with block-specific controls and clear grouping.
- Status header: Chronicle name, draft state, save state, validation state, preview, version/publish state.
- Context shelf: transient tools relevant to current selection or multi-selection; disappears when not useful.

No region may become a permanent junk drawer. If a control is rarely needed, it should live behind an Advanced disclosure, command palette, contextual menu, or secondary panel instead of consuming permanent layout area.

8. Authoring Modes and Progressive Disclosure

Shipwright should support three disclosure levels without creating three separate editors: Guided, Detailed, and Engineering. They are presentation modes over the same canonical draft and contracts.

Mode	Audience	Behavior
Guided	New and occasional Creators	Natural-language labels, recommended defaults, minimal required fields, contextual tips, warnings explained in plain language.
Detailed	Experienced Creators	Full block-specific options, visibility/completion/presentation controls, reusable references, branch and provider details.
Engineering	Advanced diagnostics	Schema/version identifiers, exact variable scopes, rule IDs, provider versions, raw but safe structured views, detailed Drydock traces.

Switching modes must never change data merely by revealing or hiding controls. Mode changes are view preferences. A field hidden in Guided mode remains intact and accurately summarized.

9. Chronicle Canvas and Graph Editing

The Chronicle Canvas is the primary structural authoring surface. It should represent chapters, Story Blocks, choices, conditions, optional routes, side content, and terminal outcomes without requiring the Creator to memorize target IDs.

- Smooth pan and zoom with fit-to-Chronicle and fit-to-selection.
- Optional grid, snapping, alignment guides, and spacing guides.
- Stable node positions that do not jump after ordinary edits.
- Connection handles with explicit edge semantics and readable labels.
- Collapsed chapter groups and focused chapter mode.
- Minimap for large Chronicles with current viewport and issue density.
- Branch visualization that distinguishes default, conditional, optional, failure, and completion paths.
- Selection state that survives harmless re-renders and navigation.
- Automatic layout as an explicit command, never a surprise destructive rearrangement.
- Graph search that can focus a block, variable, artifact, location, or issue.

The canvas must remain a view of canonical Chronicle structure. Shipwright may persist presentation-only layout metadata, but structural order and edge truth remain canonical Chronicle data under Drydock/One Voyage contracts.

10. Selection, Manipulation, Grouping, and Layout

- Single select, additive multi-select, range select, and lasso where practical.
- Move selected blocks together while preserving their internal relationships.
- Duplicate one or many blocks with deterministic remapping of internal references.
- Copy/paste within and across compatible Chronicle drafts with safe ID remapping.
- Move to chapter, duplicate to chapter, extract to reusable fragment, and convert to template where supported.
- Align left/right/top/bottom, distribute horizontally/vertically, and apply clean spacing.
- Collapse/expand chapters or logical groups without mutating authored behavior.
- Keyboard nudge with larger-step modifier and announce resulting position.
- Accessible move dialog as a complete alternative to pointer dragging.

Grouping is an authoring convenience unless a domain contract explicitly defines behavioral grouping. A visual group must not accidentally become a runtime condition or execution scope.

11. Commands, Toolbars, Menus, and Shortcuts

Shipwright deliberately reduces permanent button density. The UI should expose a small number of always-available primary actions and move secondary actions into predictable contextual locations.

Surface	Owns
Global header	Save/readiness state, Preview, Validate, Publish, Chronicle-level menu
Selection toolbar	Duplicate, move, group, enable/disable, delete, context-specific actions
Context menu	Less frequent structural actions on block/chapter/edge
Command palette	Searchable actions, navigation, insertion, view switching, validation commands
Inspector footer	Apply block-specific destructive or derived actions where necessary
Keyboard shortcuts	Fast access to common commands without becoming mandatory knowledge

Every command has a canonical ID, label, availability predicate, shortcut if appropriate, permission requirement, undo policy, and analytics/privacy classification if telemetry is later collected. Command availability must be derived from state, not from visually disabling controls while still accepting unauthorized server calls.

12. Inspector Architecture

The Inspector is block-aware, context-aware, and contract-driven. It should not render one generic list of arbitrary fields for every block and expect the Creator to infer meaning.

- Content: human-facing narrative, labels, prompts, choices, instructions, or media.
- Behavior: branching, visibility, recipients, provider selection, timing, interaction rules.
- Completion: success conditions and downstream targets, sourced from canonical Drydock contracts.
- Presentation: layout, reveal style, media behavior, Lanternwake scene references where appropriate.
- Accessibility: descriptions, transcripts, captions, alternatives, interaction equivalence.
- Advanced: schema/version details, precise identifiers, expert-only settings.

The inspector may still be generated from field metadata where that produces good UX, but high-value block types should receive purpose-built components. A generic fallback remains necessary for forward compatibility and unknown future adapters.

13. Block-Specific Authoring Experience

Block family	Purpose-built editor examples
Narrative	Rich text, media placement, pacing cues, Player visibility, emphasis, preview
Choice	Choice cards, branch targets, conditions, consequences, validation of duplicate or missing targets
Artifact Reveal	Artifact browser, recipient policy, reveal style, fallback representation, lore and collection impact
Location / Route	Map-based placement or selector supplied by Landfall, radius/confidence summary, fallback policy
Timer / Wait	Human-readable duration, scheduling policy, skip/fallback behavior, virtual-time preview
Condition	Visual expression builder with typed operands and readable summary
Variable	Declared variable selector, typed operation, scope and usage preview
Finale	Requirements, unlock state, ceremony preview, terminal outcome summary

Block editors must show both the current configured state and the effective behavior after defaults. Creators should not need to inspect JSON to determine whether an omitted field means false, inherited, automatic, or broken.

14. Story Block Family Expansion

Shipwright governs the experience through which new Story Block types are discovered and edited, but new semantic block types are not created unilaterally by Shipwright. Every new type requires a Drydock versioned schema and, where applicable, an owning domain adapter.

- Narrative family: rich narrative, dialogue, letter, journal entry, character message, quote, timeline.
- Interaction family: choice, multi-select, ranking, slider, checklist, matching, sequence puzzle, voting.
- Media family: image gallery, audio scene, video, ambient audio, before/after, panorama, document/parchment, interactive artifact.
- World family: Living Chart, location arrival, route, region, compass/bearing, Vision Waypoint, environmental cue.
- Logic family: condition, variable operation, counter, inventory requirement, composite requirement, random/weighted branch, cooldown, schedule.
- Crew family: Captain approval, all-ready, quorum, individual objective, shared decision, private information, role assignment.
- Ceremony family: artifact reveal, chapter unlock, major discovery, finale, celebration, Keepsake moment.

Semantic Boundary
This catalog is a product target, not permission for Shipwright to invent runtime behavior. A block becomes real only when its owning project and Drydock contract exist, validate, preview, publish, and remain historically compatible.

15. Logic, Variables, Conditions, and Branch Authoring

Once Drydock Phase 1 lands, Shipwright should present its typed variable registry and expression AST through visual, human-readable authoring rather than freeform identifiers and opaque JSON values.

- Variable browser grouped by scope and type, showing default, writers, readers, and unused state.
- Rename workflow with Drydock-provided reference propagation and preview of affected blocks.
- Visual condition builder with typed left operand, operator, right operand, and nested AND/OR groups.
- Readable summary sentence beside the exact structured expression.
- Branch target picker that shows chapter, block title, reachability, and consequences instead of raw IDs.
- Impossible or contradictory conditions highlighted through Drydock issues, not reimplemented locally.
- Random/weighted behavior shown with explicit probabilities and deterministic preview seed when supported.
- Inventory/artifact conditions select canonical definitions rather than manually typed names.

16. Chapters, Templates, Fragments, and Reuse

Shipwright should make reusable composition first-class. Creators frequently repeat structural patterns: introductions, clue sequences, choice branches, timed pauses, artifact ceremonies, and finales. These should be reusable without copying brittle IDs by hand.

- Chapter templates with named entry and exit points.
- Reusable block fragments/subgraphs with safe namespacing and remapped references.
- Personal Studio templates and installed Harborlight templates.
- Duplicate Chronicle or derive-from-template workflows that preserve attribution and source identity where required.
- Template parameter slots for human-facing fields and references, not arbitrary code.
- Usage inspection before updating or deleting reusable private templates.
- Preview of what will be inserted before the draft mutates.

Community-installed content follows Harborlight lineage and licensing rules. Shipwright may provide the insertion UX, but it must not strip attribution or silently detach lineage.

17. Asset and Media Authoring

The asset experience should become contextual rather than a separate warehouse the Creator repeatedly leaves the editor to search. The current asset filters are useful foundations and should be preserved while adding tighter block-specific selection.

- Search by name, description, tag, role, media type, collection, usage, and authoring context.
- Recent and currently-used views with usage counts and “used by” navigation.
- Asset-picker constraints derived from the selected field contract, such as image-only, transcript-required, or poster-required.
- Preview states for processing, scanning, unavailable variants, private access, and broken media.
- Drag or choose asset into a compatible field without exposing raw storage keys.
- Safe replacement showing every current usage before a destructive change.
- Automatic accessibility prompts for alt text, captions, transcript, poster, or fallback when contracts require them.

18. Location, Artifact, Provider, and Library Integration

Studio should not hard-code every future subsystem into TaleEditor. Domain projects register authoring adapters that describe browse cards, picker requirements, inspection panels, current state, validation summary, and allowed actions.

Provider	Shipwright experience
Landfall	Map/location/route browser, embedded map authoring, arrival/fallback summary
Watchglass	Vision Waypoint library, exact version selection, certification/compatibility summary, test launch
Artifact / Relic systems	Artifact browser, representation preview, assembly and reveal configuration
Sealed Hold	Private asset state, scan/quarantine status, protected preview availability
Harborlight	Installed content library, attribution, license, update/lineage status
Future providers	Versioned adapter registration, no generic unvalidated JSON panel

19. Preview and Player/Captain Perspective

Preview must answer “what will this feel like?” without pretending to be a live Voyage. It should use the canonical published-block rendering and Drydock-compatible draft preview contract while preserving clear draft status.

- Preview selected block, chapter, or whole draft from a chosen starting point.
- Desktop/mobile/tablet viewport presets and resizable custom preview frame.
- Full, gentle, product-reduced, and browser-reduced motion modes where relevant.
- Player-safe projection by default; Captain preview is explicitly separate and privileged.
- Audio on/off, transcript view, unavailable-media fallback, and selected failure states.
- Role/player-count context where block behavior is member-specific.
- Replay of presentation without mutating draft or authoritative runtime state.
- Clear boundary between preview and Drydock simulation: preview renders; Drydock proves behavior.

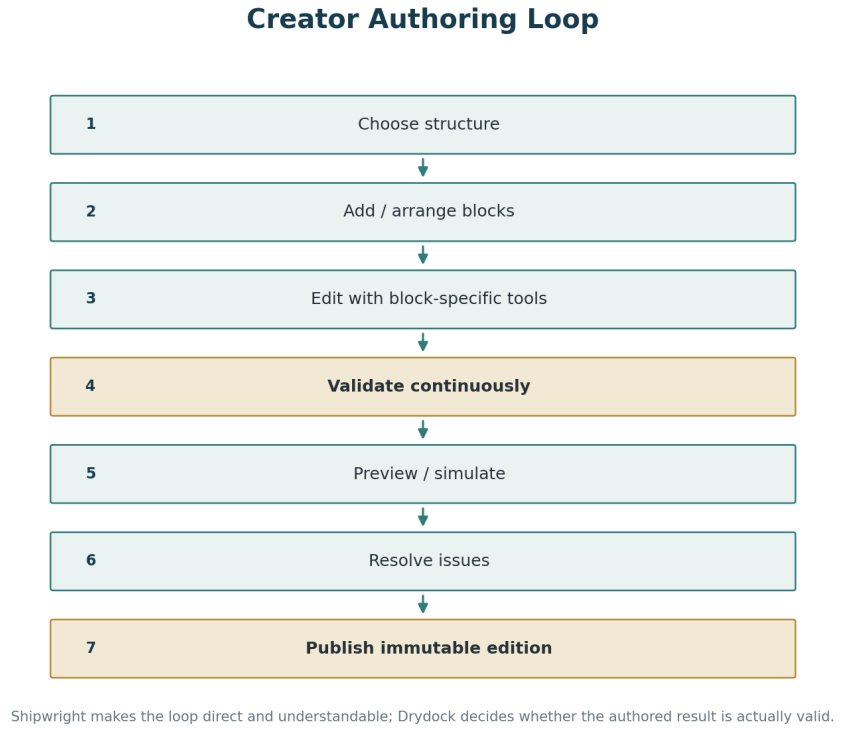

Figure 4. The normal authoring loop keeps validation and publication tied to canonical authorities.

20. Drydock Validation and Issue Integration

Drydock is the sole validation authority. Shipwright turns its issues into a usable Creator experience.

- Persistent readiness indicator showing blocker/error/warning/info counts.
- Issue list grouped by chapter, block, field, contract, or severity.
- Clicking an issue focuses the exact block/field/edge where possible.
- Inline field issue summaries with stable rule code and plain-language explanation.
- Graph overlays for unreachable nodes, cycles, missing targets, or state-flow concerns.
- Safe repair suggestions surfaced only when Drydock declares a reversible repair.
- Waiver workflow, where permitted, showing reason, scope, expiration, and publication impact.
- Stale validation clearly invalidated after edits; old green state may not remain cosmetically reassuring.

21. Scenario and Simulation Experience

After Drydock Phase 3, Shipwright becomes the primary Creator-facing scenario laboratory without owning the simulator itself.

- Create or select named scenarios such as correct answer, wrong answer, Captain approves, provider unavailable, reconnect, or offline.
- Run scenario against current draft snapshot with deterministic seed and virtual time.
- Show current block, variables, inventory, crew state, events, provider state, and assertions in a readable timeline.
- Compare two runs or compare expected versus actual final state.
- Navigate from a failed assertion directly to the relevant authoring surface.
- Coverage overlays for branches, endings, variables, providers, and failure paths.
- Cancel long runs and preserve useful bounded evidence without corrupting draft state.

Shipwright must never write simulator state into a live TaleSession or treat a simulation pass as a live-player success record.

22. Version History, Comparison, and Tideglass Integration

The current Studio already exposes version records and structured comparisons. Shipwright should turn this into a readable Creator timeline while Tideglass becomes the semantic difference authority shared with Players.

- Version timeline with current, historical, active-session usage, release notes, checksum/readiness status.
- Compare any two immutable editions without editing either.
- Human-readable categories such as narrative, structure, branching, artifacts, locations, media, accessibility, requirements, and metadata.
- Creator release notes shown beside computed differences, not substituted for them.
- “Compare draft to current published” before publication.
- Warning when active Voyages remain pinned to an older edition.
- Tideglass adapter for semantic summaries when its contract is available; no duplicate diff semantics in Shipwright.

23. Publishing and Release Experience

Publishing is a staged workflow over Drydock and One Voyage authority, not a celebratory button that fires before the database has done anything.

1. Save and freeze the candidate draft snapshot.
2. Run required Drydock validation and scenario policy.
3. Resolve blockers and review warnings/waivers.
4. Review exact draft-to-current differences and release notes.
5. Review assets, accessibility, provider compatibility, and private-content readiness.
6. Confirm immutable publication intent.
7. Invoke canonical publication service.
8. Show success only after the immutable published edition and checksum exist.
9. Offer next actions: preview edition, create Voyage, Community publish if eligible, compare versions.

If a publication operation fails, Shipwright returns to a recoverable review state with the draft intact. It must not display a successful seal, ribbon, or ceremony until authoritative publication succeeds.

24. Autosave, Undo/Redo, Conflict, and Recovery

The current Studio already preserves local unsaved changes on server conflict and maintains bounded undo/redo state. Shipwright should formalize those behaviors into a durable editing command architecture.

- Autosave status is always visible but quiet when healthy.
- Optimistic draft revision is tied to a canonical autosave version or equivalent revision token.
- Conflicts never discard the Creator's current browser state.
- Conflict view shows current local changes, latest server state, and safe resolution choices.
- Undo/redo spans structural and field edits consistently where technically possible.
- Large or non-reversible operations use checkpoints or explicit confirmation.
- Crash/reload recovery restores last server save and may offer a protected local recovery draft if the platform supports it safely.
- Publication creates a clear history boundary; undo does not mutate an immutable published edition.

25. Accessibility and Inclusive Authoring

- Every pointer drag operation has keyboard and menu alternatives.
- Graph nodes are keyboard navigable with meaningful labels, selection state, and structural context.
- Screen readers receive movement announcements, save state, validation summaries, and modal/panel context.
- 200% zoom preserves access to Library, Canvas, Inspector, and publishing actions without clipped controls.
- High contrast and forced colors preserve state indicators without relying on color alone.
- Reduced motion removes nonessential canvas/transition movement without removing orientation feedback.
- Touch targets meet appropriate size and spacing expectations.
- Error messages identify the field and repair path rather than only using red borders.
- Audio/video authoring requires transcript/caption/fallback metadata when governed by the media contract.

26. Responsive, Touch, and Small-Screen Behavior

Desktop remains the richest editing environment, but smaller devices must not become dead-end read-only pages unless a capability is explicitly unsafe or impractical and a clear explanation is provided.

Viewport	Expected behavior
Wide desktop	Three-region Library / Canvas / Inspector workspace with full productivity controls.
Laptop / narrow desktop	Collapsible Library and Inspector, preserved canvas, compact global actions.
Tablet landscape	Two-region layout, dockable panels, touch-friendly graph movement, keyboard support.
Tablet portrait	Canvas plus modal/drawer Inspector and Library; structure editing remains possible.
Phone	Focused outline and block editing, search/navigation, validation, preview, publish review; complex freeform graph layout may use constrained structural controls instead of tiny draggable nodes.

27. Motion and Lanternwake Integration

Motion should support orientation and craft, not turn Studio into a theme park. Ordinary editor transitions use the platform Motion architecture; substantial success or ceremony scenes use Lanternwake where already governed.

- Smooth panel presence, selection focus, node insert/remove, and shared-layout movement.
- No independent transform ownership conflict between dnd-kit and Motion/GSAP on the same node.
- Publish ceremony begins only after authoritative publish success.
- Autosave or validation feedback uses restrained state motion rather than repetitive celebration.
- Reduced-motion modes preserve state visibility and focus movement.
- Animation does not delay routine editing actions or block keyboard operation.

28. Performance and Large-Chronicle Architecture

Shipwright must scale beyond demonstration Chronicles. Performance work should begin with architecture, not a final emergency pass after a 400-block Chronicle makes the browser sound like a leaf blower.

- Virtualize long libraries and lists when item counts justify it.
- Avoid re-rendering every block when one inspector field changes.
- Memoize derived indexes such as asset usage, variable usage, and graph adjacency through stable selectors.
- Canvas transforms operate independently from expensive block-content rendering where practical.
- Validation requests debounce and cancel stale work rather than queueing every keystroke.
- Autosave payloads may evolve toward patch/delta approaches if current whole-draft saves become a measurable bottleneck.
- Large Chronicle benchmarks include at least 250 blocks and representative branches, assets, and validation issues.
- Performance degradation must remain understandable: show loading/progress for expensive operations rather than freezing silently.

29. Security, Privacy, and Protected Content

- All mutations require canonical Creator authorization and CSRF protections where applicable.
- The browser receives only Creator-authorized draft/private metadata needed for the current surface.
- Protected media uses authorized preview handles; never expose raw storage credentials or private keys.
- Community-installed content preserves license and attribution metadata.
- Preview role switches do not broaden server-side access to Captain-only or private Player data.
- Engineering-mode diagnostics must redact credentials, invitation secrets, private package keys, and unrelated account data.
- Clipboard/export behavior must not silently include hidden Creator notes or secret fields when copying public-facing content.
- Autosave and local recovery must respect private-content retention and device-sharing risk.

30. Testing and Acceptance Matrix

Area	Minimum acceptance evidence
Graph editing	Pointer + keyboard move, multi-select, cross-chapter move, copy/paste, undo/redo, large graph
Inspector	Purpose-built editors, generic fallback, validation focus, mode switching, field persistence
Autosave	Success, network failure, conflict, retry, refresh recovery, no data loss
Libraries	Search/filter, constrained selection, usage navigation, unavailable asset/provider states
Preview	Player/Captain separation, viewport modes, reduced motion, media fallback, no runtime mutation
Drydock integration	Issue focus, stale validation, scenario launch, failure navigation, publishing gate
Publishing	Blocked, warning/waiver, success only after immutable commit, recoverable failure
Accessibility	Keyboard, screen reader semantics, 200% zoom, forced colors, reduced motion, Axe
Responsive	Desktop, laptop, tablet portrait/landscape, phone, touch
Performance	Large Chronicle interaction, list virtualization, validation/autosave responsiveness
Security/privacy	IDOR denial, role separation, protected media, redacted diagnostics

Automated proof is necessary but not sufficient for final product acceptance. Major Studio phases must end with a real Creator walkthrough that builds, validates, previews, repairs, and publishes a synthetic Chronicle through visible controls.

31. Implementation Phases and Mainline Integration Architecture

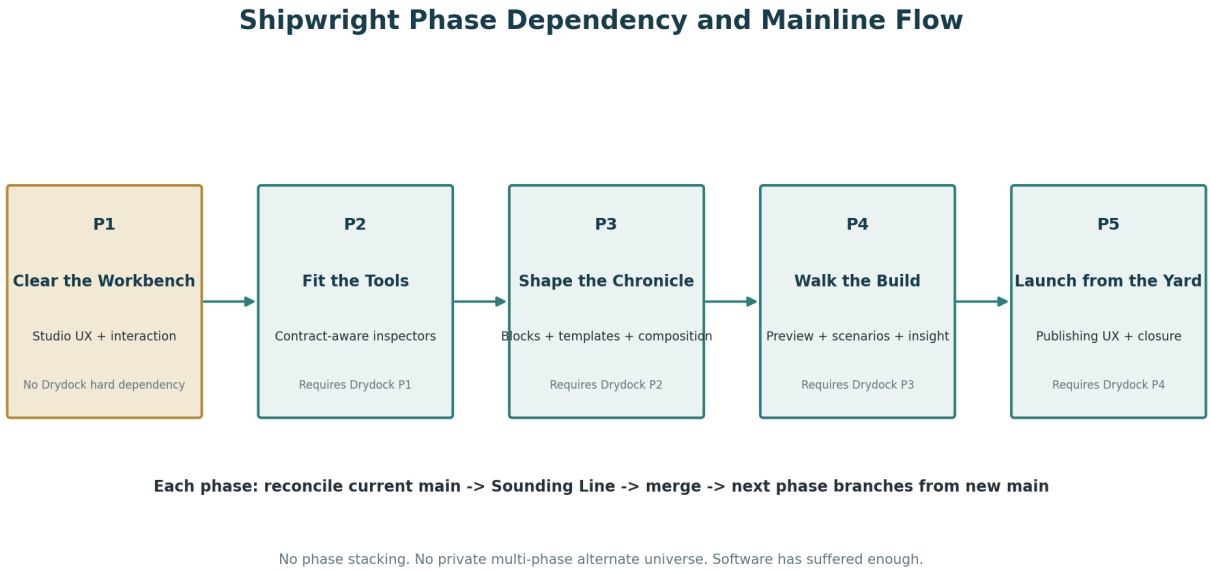

Figure 5. Shipwright is deliberately staggered with Drydock so UX can begin immediately without competing for semantic ownership.

Phase	Name	Formal scope	Hard dependency
1	Clear the Workbench	Studio Interaction, Information Architecture, Canvas Foundations, Commands, Layout, and Component Decomposition	Current accepted main only; may run beside Drydock P1
2	Fit the Tools	Contract-Aware Block Editing, Inspector Rebuild, Visual Logic, Inline Guidance, and Drydock Contract Consumption	Drydock P1 accepted in main
3	Shape the Chronicle	Expanded Block Families, Templates, Fragments, Composition, and Provider-Aware Libraries	Drydock P2 accepted in main
4	Walk the Build	Preview, Scenario Laboratory, Simulation/Issue Visualization, Version Insight, and Coverage UX	Drydock P3 accepted in main
5	Launch from the Yard	Publishing Experience, Drydock P4 Integration, Accessibility, Performance, Product Polish, and Program Closure	Drydock P4 accepted in main

Each phase receives a fresh branch/worktree from current accepted origin/main. No Shipwright Phase N+1 begins on an unmerged Phase N branch. If another project lands while a Shipwright phase is active, the phase reconciles against current main before acceptance and reruns only Sounding Line evidence invalidated by the change.

31.1 Phase 1 - Clear the Workbench

Phase 1 is intentionally safe to run in parallel with Drydock Phase 1. It may reorganize Studio components, interaction architecture, graph presentation, selection, layout, command surfaces, and responsive structure, but it may not redefine Story Block contract semantics.

- Decompose the monolithic TaleEditor into cohesive state/services/components without behavior regression.
- Establish Studio workspace shell, Library/Canvas/Inspector layout, selection model, command registry, and accessibility model.
- Improve movement, pan/zoom, snapping/alignment foundations, keyboard parity, focus restoration, and contextual actions.
- Preserve autosave, undo/redo, asset/library, preview, version, validation, and publishing behavior through compatibility adapters.
- Prefer no Prisma changes. Persist only presentation-layout metadata if justified and isolated.

Phase 1 Mainline Safety Contract
If Shipwright stopped forever after Phase 1, Creator Studio would still author the same canonical Chronicle contracts and publish the same way; it would simply have a cleaner, more modular and more usable interaction layer. Drydock may continue independently.

31.2 Phase 2 - Fit the Tools

Phase 2 begins only after Drydock Phase 1 is accepted. Shipwright replaces generic field-heavy authoring with contract-aware editors powered by the canonical schema, variable, expression, and migration contracts.

- Purpose-built inspector sections and contract-driven field components.
- Visual variable selector and operation editor.
- Visual condition/expression builder.
- Target selectors with readable destinations and validation state.
- Inline Drydock issue summaries and field navigation.
- Guided/Detailed/Engineering disclosure modes.
- No duplicate schema or validation rules in Studio.

Phase 2 Permanent-Stop Test
Main remains valid if no later phase occurs: all current blocks are easier to author and validated through Drydock, while the existing supported block set remains fully publishable.

31.3 Phase 3 - Shape the Chronicle

Phase 3 expands what Creators can build and reuse, but only through governed Drydock contracts and owning-project adapters.

- Introduce approved new block families in coherent batches.
- Add chapter templates, reusable subgraph fragments, block presets, and safe cross-Chronicle copy/import.
- Upgrade provider-aware libraries and context-sensitive insertion.
- Preserve attribution, lineage, version identity, and historical compatibility.
- Every new semantic block ships with schema, validation, preview, publication, fallback, accessibility, and compatibility evidence.

Phase 3 Permanent-Stop Test
Every block and template landed in main is independently complete. No placeholder block may appear in the library merely because a future project might eventually implement it.

31.4 Phase 4 - Walk the Build

Phase 4 turns Drydock simulation and evidence into first-class Creator tooling.

- Scenario workspace, virtual-time controls, deterministic seeds, fault choices, trace timeline, and assertion results.
- Coverage overlays and graph issue visualization.
- Role/device/motion preview matrix.
- Version timeline and semantic difference integration, using Tideglass when available.
- One-click navigation from scenario/validation failure to exact authoring location.

Phase 4 Permanent-Stop Test
Studio remains a fully usable authoring and verification environment even if the final publishing-polish phase never occurs; simulation is useful independently of final closure.

31.5 Phase 5 - Launch from the Yard

Phase 5 closes the program after Drydock Phase 4 establishes final publishing evidence and gates.

- Final staged publication workflow over Drydock readiness and One Voyage immutable publishing.
- Harborlight Community handoff where eligible.
- Final accessibility, responsive, keyboard, touch, reduced-motion, privacy, security, and large-Chronicle performance proof.
- Creator onboarding/help, command discoverability, empty/error/loading states, and polished recovery experiences.
- Complete documentation, phase receipts, Feature Catalog update, route/navigation audit, and owner walkthrough.

Program Closure Rule
Shipwright is not complete because Studio has more features. It is complete when a Creator can efficiently build a nontrivial Chronicle without learning internal IDs, fighting the interface, or bypassing the intended UI to make the system do what its backend already supports.

32. Cross-Project Dependencies and Governance

Dependency	Type	Shipwright rule
Drydock P1 -> Shipwright P2	Hard	Typed schemas/variables/expressions must be accepted before contract-aware editors become authoritative.
Drydock P2 -> Shipwright P3	Hard	New semantic block expansion requires governed static validation.
Drydock P3 -> Shipwright P4	Hard	Scenario UX consumes the accepted simulator.
Drydock P4 -> Shipwright P5	Hard	Final publication UX consumes authoritative readiness evidence.
Tideglass P1 -> Shipwright version insight	Soft until available	Use semantic diff contract when present; preserve current compare until then.
Landfall / Watchglass / artifact systems	Adapter dependency	Only expose provider-specific editors after accepted contracts exist.
Harborlight	Soft/adapter	Install/publish UX consumes Harborlight lineage and policy; no Community duplication.
Deepwater	Finding feed	Shipwright remediates Studio realization gaps assigned to it; Deepwater does not become a second Studio owner.

33. Final Acceptance Criteria

10. Creator Studio uses one canonical authoring model and Drydock validation authority.
11. A Creator can build a linear Chronicle without advanced-mode disclosure or raw IDs.
12. A Creator can build a branching Chronicle through visual target and condition tools.
13. Graph operations support pointer, keyboard, and non-drag alternatives.
14. The interface no longer depends on one monolithic component owning unrelated concerns.
15. Autosave, undo/redo, conflict handling, and reload recovery preserve work.
16. Every current and newly accepted block has a coherent editor or safe generic fallback.
17. Validation issues navigate to exact authoring context and never remain falsely green after edits.
18. Preview and simulation clearly distinguish presentation from authoritative runtime truth.
19. Publishing succeeds visually only after Drydock/One Voyage authoritative success.
20. Asset/provider libraries respect ownership, privacy, scan state, attribution, and compatibility.
21. Desktop, tablet, phone, keyboard, touch, zoom, forced-colors, and reduced-motion acceptance pass.
22. Large representative Chronicles remain usable under documented performance budgets.
23. Every phase reached main independently under the Continuous Development Standard.
24. Final owner walkthrough demonstrates creation, editing, validation, preview, recovery, version comparison, and publication without manual URL or database intervention.

34. Recommended Technical Baseline

Area	Recommended baseline
UI	React 19 / current Next.js architecture; decompose TaleEditor into cohesive Studio modules
Drag / manipulation	dnd-kit remains primary for reorder/drag ownership unless measured limitations justify a governed replacement
Presence/layout motion	Motion for editor presence and layout; Lanternwake for substantial governed scenes
Authoring contracts	Drydock typed schemas and adapters; no Shipwright duplicate schema layer
State	Local draft editing state with explicit command/history model; server remains canonical persisted draft
Autosave	Optimistic revision token/version with conflict-preserving failure behavior
Graph	Single canonical Chronicle graph with presentation-only layout metadata
Validation	Drydock incremental/full issue APIs
Simulation	Drydock scenario/simulation services
Publishing	One Voyage immutable publish service gated by Drydock evidence
Testing	Sounding Line-owned focused, browser, accessibility, performance, and mainline gates

35. Glossary and References

Term	Meaning
Chronicle Canvas	Shipwright structural view of the canonical authored Chronicle graph.
Inspector	Contextual editing surface for the selected Chronicle entity.
Library	Searchable source of blocks, chapters, templates, assets, locations, artifacts, and providers.
Guided Mode	Simplified disclosure over the same canonical authoring data.
Engineering Mode	Advanced diagnostic and exact-contract disclosure; not a different data model.
Fragment	Reusable authored subgraph with governed inputs, outputs, and remapping.
Authoring Adapter	Versioned UI integration supplied by a domain-owning project.
Mainline Safety Contract	Required statement proving a phase can live safely in main without future phases.
Drydock Issue	Authoritative authoring/validation finding produced by Project Drydock.
Semantic Edition Difference	Human-meaningful version difference produced by Tideglass when available.

References and Source Baseline

- Current repository main reviewed at f1c2f22dd935322c1a71eb80c51592f243dc196d.
- Current Creator Studio implementation, including src/components/studio/TaleEditor.tsx and related tests/components.
- Current Chronicle block registry and canonical Chronicle contracts under src/chronicle/.
- Project Drydock Governing Document v1.0, especially canonical ownership, schema evolution, cross-project boundaries, and implementation phases.
- Project Harborlight Governing Document, Creator Studio publication/install/lineage boundaries.
- Project Watchglass Part I, reusable library and guided authoring patterns.
- Voyagewright Continuous Development and Mainline Integration Standard v1.0 and the phase-level mainline doctrine adopted for all new projects.
- Project Feature Catalog review and Shipwright project concept established August 8, 2026.

Appendix A - Candidate Expanded Story Block Catalog

Family	Candidate types
Narrative	Rich Narrative, Dialogue, Letter, Journal Entry, Character Message, Quote, Timeline
Interaction	Choice, Multi-select, Ranking, Slider, Checklist, Matching, Sequence Puzzle, Voting
Media	Image Gallery, Audio Scene, Video, Ambient Audio, Before/After, Panorama, Document/Parchment, Interactive Artifact
World	Living Chart, Location Arrival, Route, Region, Compass/Bearing, Vision Waypoint, Environmental Cue
Logic	Condition, Variable Operation, Counter, Inventory Requirement, Composite Requirement, Random Branch, Weighted Branch, Cooldown, Schedule
Crew	Captain Approval, All Players Ready, Quorum, Individual Objective, Shared Decision, Private Information, Role Assignment
Ceremony	Artifact Reveal, Chapter Unlock, Major Discovery, Finale, Celebration, Keepsake Moment

Every candidate requires a governing semantic owner and Drydock contract before it can be exposed as an active block. The catalog exists to plan the authoring experience, not to authorize implementation-by-dropdown.

Appendix B - Keyboard and Command Model

Action	Recommended command / behavior
Open command palette	Ctrl/Cmd + K
Undo / Redo	Ctrl/Cmd + Z / Shift + Ctrl/Cmd + Z
Duplicate selection	Ctrl/Cmd + D
Delete selection	Delete/Backspace with confirmation rules
Search / focus block	Ctrl/Cmd + F within Studio search mode
Move selection	Arrow keys; modifier for larger increments
Open inspector	Enter or dedicated command
Close inspector / modal	Escape with focus restoration
Fit Chronicle / selection	Command palette and toolbar controls
Insert block	Keyboard-accessible library or command palette
Move to chapter	Context command with destination picker
Validate current context	Command / toolbar action; Drydock-owned semantics
Preview current context	Command / toolbar action
Publish	No single unguarded shortcut; opens staged review

Appendix C - Mainline Safety Matrix by Phase

Phase	Activation after merge	Expected schema impact	Permanent-stop result
P1	Active Studio UX improvements	Prefer none; presentation metadata only if justified	Same Chronicle semantics, cleaner modular Studio
P2	Active contract-aware editors	No competing Prisma authority; consumes Drydock	Current blocks easier and safer to author
P3	Active only for fully governed new blocks/templates	Potential authoring metadata only through owning project/Drydock	Every exposed new type is individually complete
P4	Active scenario/verification workspace	Scenario persistence remains Drydock-owned	Studio gains useful executable proof without final closure
P5	Active final publishing/workflow polish	Minimal/none; release data remains canonical owners	Full program closure and owner acceptance

Appendix D - Cross-Project Ownership Matrix

Concern	Canonical owner	Shipwright responsibility
Block schema / defaults / migration	Drydock	Render/edit canonical contract
Validation rules / issues	Drydock	Explain and navigate issues
Simulation / virtual time	Drydock	Creator-facing scenario UX
Draft persistence / immutable publishing	One Voyage / Chronicle platform	Invoke and present canonical operations
Asset protection / scanning	Sealed Hold	Preview states and selection UX
Community install / lineage / licensing	Harborlight	Insert/configure while preserving metadata
Location and route semantics	Landfall	Map/provider authoring adapter
Vision verification semantics	Watchglass	Waypoint selection/configuration adapter
Artifact runtime/rendering/ownership	Artifact/Wayfarer domains	Definition/reveal authoring UX only
Semantic version differences	Tideglass	Creator compare presentation
Animation runtime	Lanternwake	Use governed scene/motion contracts
Testing authority	Sounding Line	Supply testable surfaces and respect gates

Appendix E - Creator Studio State and Failure Matrix

State	Required experience
Loading	Skeleton/structure; no fake empty Chronicle
Loaded clean	Saved state visible; normal editing enabled
Unsaved	Quiet explicit indicator; autosave scheduled
Saving	Nonblocking status; edits remain possible
Save failed	Changes preserved locally; retry and explanation
Conflict	Local work preserved; compare/reload/resolve options
Validation stale	Previous result visibly stale after edits
Validation blocked	Exact issues and navigation; publish unavailable
Preview unavailable	Readable explanation and fallback
Provider unavailable	Provider-specific truthful state; no fake completion
Publishing	Staged operation; destructive editing appropriately constrained
Publish failed	Draft intact; recoverable review state
Published	Immutable edition identity and next actions
Offline / weak network	No false saved state; bounded local behavior and recovery guidance

Appendix F - Program Closure Checklist

25. All five Shipwright phases are individually present in accepted mainline history.
26. No Shipwright phase was intentionally stacked on an unmerged predecessor.
27. Drydock remains the sole authoring-contract, validation, and simulation authority.
28. One Voyage remains the sole runtime and immutable publication authority.
29. Creator Studio has no ordinary user workflow requiring raw IDs or manual JSON entry when a governed visual control exists.
30. Every drag operation has a complete non-drag alternative.
31. Every active block type has an accessible, usable editor and preview/fallback path.
32. All new block types have historical compatibility and Drydock evidence.
33. Autosave/conflict/recovery tests prove no silent data loss.
34. Desktop, tablet, phone, keyboard, touch, 200% zoom, forced colors, and reduced-motion evidence are current.
35. Large Chronicle performance evidence meets the adopted budget or has an approved visible limitation.
36. Security/privacy review finds no private asset keys, cross-role leakage, or unauthorized mutation path.
37. Publishing success is tied to authoritative Drydock/One Voyage success.
38. Feature Catalog and project-status records describe implemented truth.
39. Owner walkthrough completes a nontrivial Chronicle authoring journey and grants PRODUCT ACCEPTED status.

Final Governing Rule
Shipwright succeeds when power becomes easier to use, not merely when more power exists. Creator Studio must reveal the capabilities of Voyagewright without exposing the machinery as a tax on every Creator.